WDYRI: Shortcuts, Leakage and Transfer in Swahili Scam SMS Detection

Shima Byiringiro Serein

October 2026

Institution:

African Leadership University

Public GitHub Repository:

https://github.com/supserrr/wdyri

Demo Video:

https://youtu.be/-evEAtiSujw

Live System:

https://huggingface.co/spaces/supserrr/wdyri

Existing resources and my own work: The two datasets (BongoScam and the Chichewa SMS fraud set), the pretrained models (XLM-R base, AfroXLMR base and the fastText Swahili vectors) and the libraries are existing resources, cited where they are used. I built everything else: the data pipeline (cleaning, masking, template detection and the leakage-safe splits), the training and tuning code for every model, the rule-based attacks, the defences, the minimal-pair stress tests, number-balanced training, the ensemble, the evaluation and significance tests, and the browser application (src/, scripts/ and web/ in the repository).

Abstract

Published Swahili scam-SMS detectors report 98.7% to 99.86% accuracy, but they are tested on random splits of a single dataset. This project tests whether those scores survive harder evaluation. It classifies Tanzanian Swahili SMS as scam or not scam using the public BongoScam dataset (1,064 messages after cleaning, 567 scams), and tests transfer on 733 Chichewa SMS from Malawi. A ladder of models, from rule baselines and Naive Bayes (the published baseline) through character n-gram logistic regression and a BiLSTM over fastText vectors to fine-tuned XLM-R and AfroXLMR, is evaluated on template-disjoint splits with rule-based obfuscation attacks, control sets, minimal-pair stress tests and paired significance tests. The published accuracy reproduces (98.68%), but 86% of scams and no genuine messages contain a phone number or link, and appending a number to a genuine message makes fine-tuned AfroXLMR call it a scam 96% of the time. Number-balanced counterfactual training removes this shortcut: those false alarms fall to 0%, and misses on scams stripped of their number fall from 20% to 2%. An ensemble of the character n-gram model and AfroXLMR, both number-balanced, scores test F1 0.994 and keeps 0.95 to 0.99 under lookalike-letter and split-word attacks. Zero-shot transfer to Chichewa is weak (fraud F1 0.46 to 0.75), but 20 Chichewa examples lift the character model from 0.65 to 0.82. Near-perfect in-language scores therefore overstate what these detectors learn. The deployed ensemble runs entirely in the browser.

1. Introduction

Mobile-money scams reach their victims as text messages. They pose as a relative in trouble, an agent who sent money by mistake, a landlord with a new number or an employer, and they ask the reader to send money. In Tanzania a common hook is *"ni tumie kwa namba hii"* (send it to me on this number), and smishing aimed at mobile-money users has been studied as a Swahili text classification problem (Mambina et al., 2022; Dioniz, 2024a). The channel is large: Tanzania had 87.05 million mobile-money subscriptions in June 2026 and 2.1 billion mobile-money transactions in that quarter, and operators reported 7,334 fraud attempts in the same three months (Tanzania Communications Regulatory Authority, 2026).

The intended users are mobile-money customers who want to check a suspicious SMS before they pay, and the telecom operators and fintech companies that could run such a check for them. Both need a detector that recognises the language of a scam, because a scammer can change the number, the name and the spelling in every new message.

The project is text classification for Swahili (scam or not scam). Swahili is chosen because it has public, real-world scam SMS and several published detectors to test against. Swahili is well served by machine translation, but this project does not translate, so its case rests on the task: no established system classifies Swahili scam SMS, and the published detectors have only been evaluated on random splits or random cross-validation (section 2). Chichewa, a related Bantu language that XLM-R did not see in pretraining (Conneau et al., 2020), extends the question to a lower-resource neighbour.

Published detectors report 98.7% (Dioniz, 2024b) to 99.86% (Mambina et al., 2022) accuracy. This project therefore asks three questions:

RQ1, leakage. How much of the score survives when near-duplicate scam templates cannot appear in both training and test data?

RQ2, robustness. How far does each model fall under rule-based obfuscation (lookalike letters, split words, English code-switching), and do simple defences recover it?

RQ3, transfer. Can a model trained only on Swahili detect real Chichewa fraud SMS, and does Africa-centric pretraining (AfroXLMR) beat plain XLM-R there?

Answering them raised a fourth question that became the main finding: what are these models actually reading? The objective is a scam detector whose scores hold up under leakage-safe, adversarial and cross-language tests, deployed as a web application. The work contributes a measured phone-number shortcut in the benchmark data and minimal-pair tests that expose it; a targeted counterfactual fix; leakage-safe evaluation with repeated splits and significance tests; reproducible attacks with controls, a second attacker and a held-out defence test; a complementary ensemble, prompted by test errors but with its members chosen on validation data and re-checked on fresh splits; and a Swahili-to-Chichewa scam transfer study, to my knowledge the first.

2. Related Work

2.1 SMS Spam and Smishing Detection

Text-message spam filtering has long been treated as text classification; the SMS Spam Collection (Almeida et al., 2011) made English benchmarks standard, with Naive Bayes and support vector machines over word counts as strong baselines. Swahili smishing work follows the same recipe

(Table 1). Mambina et al. (2022) report 99.86% accuracy with TF-IDF features and a random forest on 32,259 Swahili messages, but, as Taylor and Robert (2025a) note, scams are under 1% of their data and the data are not public. BongoScam (Dioniz, 2024a, 2024b) released 1,508 labelled Tanzanian messages with a word-count Naive Bayes model at 98.7% accuracy. Njame et al. (2026) report F1 near 0.998 under five-fold cross-validation on 2,408 Tanzanian short messages, 1,377 of them real SMS from BongoScam; because performance held across four ablations, they conclude that the signal is linguistic rather than an artefact of how the data were built. For Swahili SMS spam more broadly, Mambina et al. (2024) report 99.98% accuracy with a CNN-LSTM hybrid under ten-fold cross-validation on telecom data. All of these evaluate on random splits or random cross-validation of one dataset, so copies of one scam script can sit on both sides. For Chichewa, Taylor and Robert (2025a) built the fraud set used here and report about 97% accuracy with classical, in-language models only.

Study	Data	Reported result	What it leaves open
Mambina et al. (2022)	32,259 Swahili SMS; scams under 1%; data not public	99.86% accuracy, TF-IDF + random forest	Accuracy on a tiny scam class; no robustness or transfer test
Mambina et al. (2024)	Swahili SMS spam from Tanzanian telecom operators	99.98% accuracy, CNN-LSTM hybrid, ten-fold CV	Spam rather than scams; random folds only
BongoScam (Dioniz, 2024a, 2024b)	1,508 Tanzanian Swahili SMS, public on Kaggle	98.7% accuracy, word counts + Naive Bayes	One random split; no leakage, robustness or transfer check
Njame et al. (2026)	2,408 Tanzanian short messages, 1,377 of them real SMS from BongoScam	F1 about 0.998 (five-fold CV), linear SVM and random forest	Random folds only; reads stable ablations as a linguistic signal
Chiuseni et al. (2026)	27,037 English-Swahili SMS, 356 scams	Classical models lost up to 98.8% of F1 under obfuscation; transformers at most 35.1%	Attacks written by a language model; defences proposed, not tested
Taylor and Robert (2025a)	824 Chichewa SMS, partly rewritten by the authors	About 97% accuracy, random forest and SVM	Classical, in-language models only

Table 1. Prior Swahili and Chichewa scam and spam SMS detectors.

2.2 Leakage and Evaluation Design

A single random split can overstate how well a model generalises. Gorman and Bedrick (2019) show that differences reported on a standard split often do not hold on randomly drawn splits, and Søgaard et al. (2021) argue that random splits overestimate performance on new data and recommend harder, biased splits. Near-duplicates shared between training and test inflate scores further (Elangovan et al., 2021). Scam SMS are prone to this because a campaign reuses one script with a new name or number. Repeated splits, bootstrap intervals and paired tests follow standard advice on significance testing in NLP (Dror et al., 2018), and several seeds per neural model reflect

how much fine-tuning varies with the seed on small data (Dodge et al., 2020; Mosbach et al., 2021).

2.3 Shortcuts and Behavioural Testing

Models often succeed by exploiting artefacts of how a dataset was built rather than the intended signal: cue words in natural language inference hypotheses (Gururangan et al., 2018), syntactic heuristics (McCoy et al., 2019) and shortcut learning more broadly (Geirhos et al., 2020). Behavioural tests with minimal edits expose such failures where aggregate accuracy hides them (Ribeiro et al., 2020), and counterfactually augmented data, in which documents are minimally revised so that their label flips, makes models rely less on spurious features (Kaushik et al., 2020).

2.4 Adversarial Text and Defences

Small character-level changes break text classifiers: gradient-guided character flips (Ebrahimi et al., 2018), visually similar Unicode characters (Eger et al., 2019) and invisible or reordering characters (Boucher et al., 2022); word substitutions also degrade fine-tuned BERT (Jin et al., 2020). Proposed defences include a word-recognition front end that repairs adversarial misspellings (Pruthi et al., 2019) and visual embeddings or adversarial training (Eger et al., 2019). For smishing, Chiuseni et al. (2026) found that classical detectors lost up to 98.8% of their F1 under obfuscation written by a language model, while transformers lost at most 35.1%; they proposed defences but did not test them.

2.5 Cross-Lingual Transfer and African Languages

Most of the world's languages, including most African languages, have little labelled data or tooling (Joshi et al., 2020). Multilingual encoders transfer a classifier trained in one language to others without target-language labels (Pires et al., 2019; Wu & Dredze, 2019; Conneau et al.,

2020), but zero-shot transfer is weakest for low-resource and distant targets, and a few target-language examples close much of the gap (Hu et al., 2020; Lauscher et al., 2020). AfriBERTa, pretrained on less than 1 GB of text in 11 African languages, is competitive with much larger multilingual models (Ogueji et al., 2021), and AfroXLMR continues XLM-R's pretraining on 17 African languages including Swahili and Chichewa (Alabi et al., 2022).

Each strand has been studied, but not together for African-language scam SMS. Published Swahili detectors stop at random splits or random cross-validation, the one robustness study used English-Swahili data and untested defences, and, to my knowledge, no earlier work tests scam detection across African languages. This project combines leakage-safe evaluation, a measured shortcut and its fix, reproducible attacks with tested defences, and Swahili-to-Chichewa transfer.

3. Dataset and Data Preparation

3.1 Sources

Two public datasets play different roles and are never mixed (Table 2): BongoScam trains, tunes and tests every model in Swahili, and the Chichewa set is used only to test transfer. BongoScam is the public dataset behind the 98.7% result this project set out to test, so results compare directly with prior work. The Chichewa set contains real Malawian fraud messages plus genuine personal and telco texts, in a language related to Swahili; its other sheets are English translations and are not used.

Dataset	Language	Raw rows	After cleaning	Role	Licence
BongoScam / Swahili SMS Detection Dataset (Dioniz, 2024a)	Swahili, Tanzania	1,508 (1,000 scam, 508 trust)	1,064 (567 scam, 497 not scam)	Training, validation, in-language test	MIT
SMS Fraud Classification dataset for Chichewa (Taylor & Robert, 2025b), sheets D_CHI and telcoSMS_CHI	Chichewa, Malawi	824 (338 fraud, 486 normal)	733 (336 fraud, 397 normal, 59 of them telco)	Zero-shot transfer test only	CC BY 4.0

Table 2. The two datasets.

3.2 Cleaning and Masking

Each step and its reason are logged in the repository's design log (DECISIONS.md).

Drop exact duplicates. 131 BongoScam rows. Copies inflate the test set and can sit on both sides of a split.

Mask numbers, amounts and links. Phone numbers, money amounts and links become <PHONE>, <AMOUNT> and <URL>, so a model cannot memorise one scammer's number and no personal number is kept in the repository.

Drop messages that become identical. 313 BongoScam rows (all scams) and 91 Chichewa rows (78 of them exact copies; 89 of the 91 are repeated telco texts); one job-scam campaign would otherwise fill a whole split with 35 copies.

Keep punctuation, casing and stop words. Removing punctuation and stop words hurt every model in the Chichewa study (Taylor & Robert, 2025a), and casing is kept because all-capital adverts are a real scam signal.

Group near-duplicate templates. Two masked messages share a template when their lower-cased character 5-grams overlap by at least 80% (Jaccard), grouped transitively. This catches the same script with a different name or network: BongoScam has 909 templates, 73 with two or more members.

3.3 Splits

Two 70/15/15 splits are made, both stratified by label: a random split that reproduces prior work, and a template-disjoint split (StratifiedGroupKFold) that keeps every copy of a script on one side. The template split (seed 42, fixed before any modelling) has 760 training, 152 validation and 152 test messages (81 test scams), and no template crosses parts; the random split leaks 40 templates across parts.

3.4 Data Quality and Limits

Undocumented provenance. The BongoScam author does not say how messages were collected or labelled. The scams carry real-looking numbers and names; the genuine texts read like composed personal chat.

Shortcut 1, numbers. None of the 497 genuine messages contains a phone number or link, while 86% of scams do. The rule "contains a phone number or link" scores F1 0.92 across all 1,064 messages with no training.

Shortcut 2, length. Every scam has at least 8 words after masking (median 12); genuine texts have a median of 7. A rule "at least 9 words" scores test F1 0.85.

One dominant test template. A landlord-impersonation script ("this is my new number") holds 31 of the 81 test scams. It is kept, because it is exactly the kind of unseen script RQ1 is about, and every test score is also reported with one message per template.

Chichewa rewrites and masking rules. The Chichewa fraud set grew from 126 original messages through label-preserving rewrites by its authors (Taylor & Robert, 2025a), and the files do not mark the originals. The masking rules were written after looking at samples of both datasets. No model, threshold or hyperparameter was tuned on the Chichewa set.

3.5 Machine-Made Data

Two experiments change the training data, and both keep it made of real messages. Adversarial training (E7) adds 101 perturbed scam copies made by the project's attack code, 12% of the augmented training set. Number-balanced training (E11) edits 42% of training rows in place, deleting or appending placeholder tokens, and changes no label or other word. No text was generated by a language model. Both licences allow redistribution, so only the masked, processed text is committed, with the licence notices.

4. Methodology

4.1 Model Ladder and Baseline

The models climb a ladder from a one-line rule to an Africa-centric transformer, so each rung's gain or loss is measured rather than assumed (Table 3). Every model sees text through the same preprocess() function that the app uses. The baseline (E1) reproduces BongoScam's published setup exactly: all 1,508 raw rows, an 80/20 split with random_state=42, and a default CountVectorizer with MultinomialNB in scikit-learn (Pedregosa et al., 2011). Naive Bayes on

word counts is the classic spam baseline (Almeida et al., 2011) and the one prior Swahili work used.

Rung	Model	Input	Why it is here
0	Majority class; phone rule; length rule	None; "contains <PHONE> or <URL>"; "at least 9 words"	Floors that any model must beat, including each shortcut
1	Multinomial Naive Bayes (baseline)	Word counts	The model BongoScam and Mambina et al. (2022) used
2	Logistic regression	Character 2-5-gram TF-IDF	Sub-word features should survive misspellings; the weights are interpretable
3	BiLSTM	fastText Swahili vectors (300-d): random, frozen or fine-tuned	The course's embeddings and recurrent core
4	XLM-R base, fine-tuned	SentencePiece sub-words	Multilingual transformer control, no Chichewa in pretraining
5	AfroXLMR base, fine-tuned	SentencePiece sub-words	XLM-R adapted to 17 African languages, including Swahili and Chichewa
+	Ensemble	Rungs 2 and 5, both number-balanced	Flags a message if either member flags it; the deployed system

Table 3. The model ladder.

4.2 Classical Models

Naive Bayes and logistic regression use word or character n-grams; logistic regression uses class_weight="balanced". The 152-message validation set is saturated (F1 0.99 to 1.00 for most settings), so hyperparameters (Naive Bayes alpha 0.001 to 10, logistic regression C 0.01 to 1000) are chosen by template-disjoint 5-fold cross-validation on the training set, which picks interior values for all four models.

4.3 BiLSTM over fastText

Tokens map to 300-dimensional fastText Swahili vectors (Bojanowski et al., 2017; Grave et al., 2018), loaded with gensim (Řehůřek & Sojka, 2010). They are built from character n-grams, so misspelt and disguised words still get a vector. A bidirectional LSTM (Hochreiter & Schmidhuber, 1997; Schuster & Paliwal, 1997) with 128 units per direction reads the packed sequence; max-pooling over time keeps the strongest signal anywhere in the message, then dropout 0.3, a linear layer and a sigmoid give P(scam). Three embedding variants test E4: random initialisation, frozen fastText vectors, and fastText vectors fine-tuned with a ten-times smaller learning rate (1e-4 against 1e-3). Training uses Adam, batch 32, up to 30 epochs and early stopping on validation F1 (patience 4).

4.4 Fine-Tuned Transformers

XLM-R base (Conneau et al., 2020) and AfroXLMR base (Alabi et al., 2022) share one architecture and size, so any difference isolates African-language pretraining. A message becomes at most 128 SentencePiece tokens, a 12-layer encoder reads them, and the vector of the first token passes through a dense layer with tanh and a linear layer to two logits, the standard classification head of Hugging Face Transformers (Wolf et al., 2020), trained with PyTorch (Paszke et al., 2019). Fine-tuning uses AdamW, learning rate 3e-5, weight decay 0.01, 10% linear warm-up, batch 16, up to 5 epochs with early stopping on validation F1 (patience 2) and class-weighted cross-entropy. No learning-rate search was run: validation F1 reaches 0.99 to 1.00 after the first epoch in every run (Figure 1), so a search would only fit noise. Every neural model is trained with three seeds (13, 42, 2026; the two fresh splits in E12 use seed 42 only) and reported as mean and standard deviation, because fine-tuning on about 760 messages varies with the seed (Dodge et al., 2020). All runs were trained on a laptop CPU, 5 to 14 minutes per transformer run.

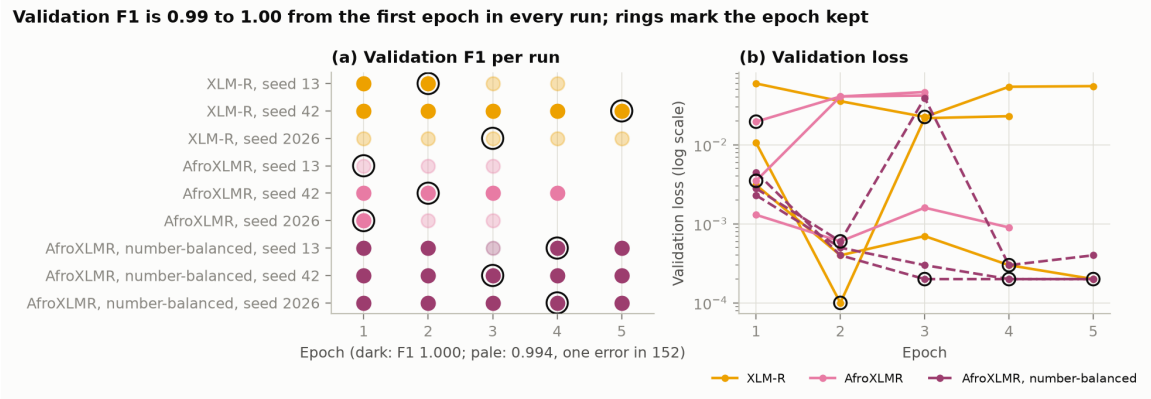


Figure 1. Validation F1 per run and epoch (a) and validation loss (b) for the nine transformer runs on the template split, from the training log; rings mark the epoch kept by early stopping.

4.5 Attacks and Defences

Three rule-based attacks change a scam's trigger words: lookalike characters (Cyrillic twins, digits for letters, a zero-width space), structural changes (spaced, dashed or joined words) and code-switching (30 Swahili scam phrases replaced by English ones). Trigger words are the words whose removal most lowers a fixed character-LR attacker's scam probability, so every model is attacked on the same words; the attack is therefore white-box only for a character LR. Each attack runs at three intensities (1, 3 or all trigger words), only on scams. Control sets disguise the genuine test texts the same way, so a model that simply flags odd-looking text is caught. A second attacker (word-count Naive Bayes) repeats the strongest attacks, and a held-out lookalike attack uses characters the defence was not written for.

Two defences are tested. Normalisation at inference maps lookalikes back to Latin letters, removes zero-width characters, undoes digit swaps and rejoins spaced letters. Adversarial training adds the perturbed copies described in section 3.5.

4.6 Shortcut Tests, Number-Balanced Training and the Ensemble

Minimal-pair stress tests (E10) append a <PHONE> or <AMOUNT> token to each genuine test text, and delete the placeholders from each test scam that has one, in the spirit of behavioural testing (Ribeiro et al., 2020). Only one token changes, so any change in the decision is caused by the number alone. Number-balanced training (E11) is a targeted counterfactual augmentation (Kaushik et al., 2020): it deletes the placeholders from half the training scams that have one and appends <PHONE> to each genuine training text with probability 0.4, so a number appears in 44% of scams and 38% of genuine texts. The ensemble (E12) flags a message if either the character LR or AfroXLMR flags it; which members it uses was chosen on validation stress pairs, never on test, although the idea of combining the two came from test errors (section 8.1).

4.7 Evaluation

Every model is judged on the scam class: precision, recall, F1 and PR-AUC, with accuracy as context only, because a detector that flags nothing still looks accurate when one class dominates. Test F1 carries a bootstrap 95% interval (1,000 resamples, seed-paired for neural models). Robustness is the relative F1 drop, (F1 clean minus F1 attacked) divided by F1 clean, the measure Chiuseni et al. (2026) used. Key differences get a paired bootstrap or, for the ten re-drawn splits, a Mann-Whitney U test. The operating threshold is the highest one up to 0.5 that catches at least 95% of validation scams; every run already met that at 0.5.

5. Experiments and Results

5.1 Overview

The published score reproduces, but on a template-disjoint test the classical models stay near 0.95 to 0.98 F1 while every neural model falls to about 0.76 to 0.82, because they lean on the phone number that one unseen scam script lacks. Twelve experiments trace this (Table 4), and Table 5 gives every model's main scores.

ID	Question	What changes	Main result
E1	Baseline	Naive Bayes on word counts, published setup	98.68% accuracy (published: 98.7%), F1 0.991
E2	RQ1 leakage	Random vs template split, once and over 10 re-drawn splits	Character models lose 2.6 to 5.7 F1 points, word models 1 point or less; all four gaps significant ($p \leq 0.02$)
E3	Features	Word vs character n-grams; masking on vs off	Word features score highest; masking changes little in Swahili
E3b	Shortcuts	Phone rule, length rule, placeholders deleted	Rules alone reach test F1 0.74 and 0.85; without placeholders classical Chichewa transfer collapses (0.65 to 0.11)
E4	Embeddings	BiLSTM with random, frozen or fine-tuned fastText vectors	0.76 to 0.77 test F1 for all three; pretrained vectors raise Chichewa F1 (0.75 vs 0.57)
E5	Transformers	XLM-R vs AfroXLMR, 3 seeds	0.76 vs 0.78 test F1 (not significant, $p = 0.39$)
E6	RQ2 attacks	3 attacks × 3 intensities; controls; a second attacker	Word LR loses 53% of F1 under full lookalike, character LR 21 to 24% under lookalike and split words; transformers lose nothing but flag 54 to 93% of fully disguised genuine texts
E7	RQ2 defences	Normalisation; adversarial training; held-out lookalikes	Normalisation cuts word LR's lookalike loss from 53% to 1% but recovers nothing on held-out characters; adversarial training cuts code-switching loss from 19% to 4%
E8	RQ3 transfer	Swahili-trained models on Chichewa	Fraud F1 0.46 to 0.75; AfroXLMR ranks fraud better than XLM-R (PR-AUC 0.81 vs 0.65)
E9	RQ3 few-shot	Add 20 or 50 Chichewa messages, same test messages	Character LR 0.65, 0.82, 0.88; AfroXLMR 0.72, 0.75, 0.87; XLM-R 0.72, 0.70, 0.78
E10	Shortcut diagnosis	A number added to genuine texts, removed from scams	False alarms with an added number: AfroXLMR 96%, XLM-R 40%, BiLSTM 28%, character LR 15%
E11	Shortcut fix	Number-balanced counterfactual training	AfroXLMR false alarms 96% to 0%, misses 20% to 2%; test PR-AUC 0.958 to 0.985
E12	Combination	Character LR OR AfroXLMR, chosen on validation; 2 fresh splits	Test F1 0.994; 0.963 under lookalike, 0.948 under split words; 0.994 and 0.988 on fresh splits

Table 4. Experiments, on the template-disjoint split unless noted.

Model	Test F1	95% interval	Test F1, one per template	Test PR-AUC	Chichewa F1	Chichewa PR-AUC
Majority class	0.695	0.62 to 0.76	0.585	0.533	0.629	0.458
Phone rule	0.744	0.66 to 0.82	0.969	0.810	0.681	0.692
Length rule	0.850	0.79 to 0.90	0.774	0.747	0.724	0.573
Naive Bayes, word counts (baseline)	0.982	0.96 to 1.00	0.971	1.000	0.608	0.648
Naive Bayes, char 2-5	0.953	0.91 to 0.98	0.926	0.991	0.683	0.738
Logistic regression, word 1-2	0.981	0.96 to 1.00	0.990	1.000	0.456	0.727
Logistic regression, char 2-5	0.968	0.94 to 0.99	1.000	1.000	0.651	0.730
BiLSTM, random init	0.772 ± 0.014	0.69 to 0.84	0.983	0.942	0.572	0.744
BiLSTM, fastText frozen	0.756 ± 0.009	0.67 to 0.83	0.977	0.890	0.752	0.751
BiLSTM, fastText fine-tuned	0.763 ± 0.000	0.68 to 0.84	0.990	0.980	0.750	0.800
XLM-R base	0.759 ± 0.007	0.67 to 0.83	0.983	0.871	0.717	0.654
AfroXLMR base	0.776 ± 0.021	0.70 to 0.84	0.990	0.958	0.719	0.805
Logistic regression, char 2-5, number-balanced	0.988	0.97 to 1.00	0.980	0.994	0.146	0.581
AfroXLMR base, number-balanced	0.820 ± 0.035	0.75 to 0.88	0.990	0.985	0.716	0.649
Ensemble, both number-balanced (deployed)	**0.994**	0.98 to 1.00	0.990	0.996	0.715	0.642

Table 5. Main results on the template-disjoint split. Scam F1 at threshold 0.5; neural models are the mean (± sd) of 3 seeds; the interval is a seed-paired bootstrap over test messages.

5.2 RQ1: Leakage and Unseen Scripts

Over ten re-drawn splits, keeping templates out of the test costs the character n-gram models 2.6 to 5.7 F1 points (character LR 0.998 to 0.972, character Naive Bayes 0.995 to 0.938) and the word models about one point or less (Figure 2). Character n-grams memorise a script's surface form, which a random split rewards. The larger effect is which scripts land in the test: the landlord script holds 31 of 81 test scams and carries no number, amount or link. Word Naive Bayes catches all 31 and word LR 29, but the transformers and BiLSTMs catch 0 to 13%, so their test F1 is 0.76 to 0.78 while their per-template F1 is 0.98 to 0.99.

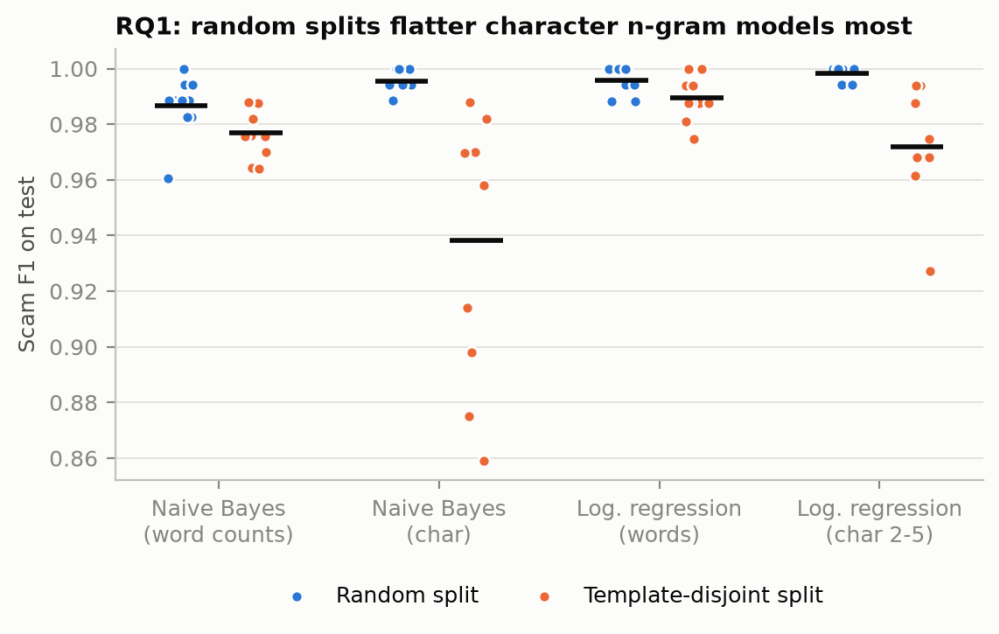


Figure 2. Test F1 over ten re-drawn random and template-disjoint splits; bars mark the means.

5.3 RQ2: Robustness to Disguise

Lookalike letters cut word LR recall from 0.96 to 0.30, and normalisation restores it to 0.95 (+0.52 F1, $p < 0.001$) at no cost on clean text (Figure 3); Figure 4 shows the same recovery for word Naive Bayes (0.62 to 1.00) and the character LR (0.58 to 0.95). That recovery is a best case: the normalisation table covers every character the attack uses, and against held-out lookalike

characters it recovers nothing (word LR still loses 45% of its F1, p = 1.0). The transformers lose nothing under any attack, but the control sets show why: with every word disguised, they flag 54 to 93% of genuine texts as well. A second attacker gives the same picture. Code-switching is the attack normalisation cannot undo; adversarial training cuts the character LR's loss there from 19% to 4%.

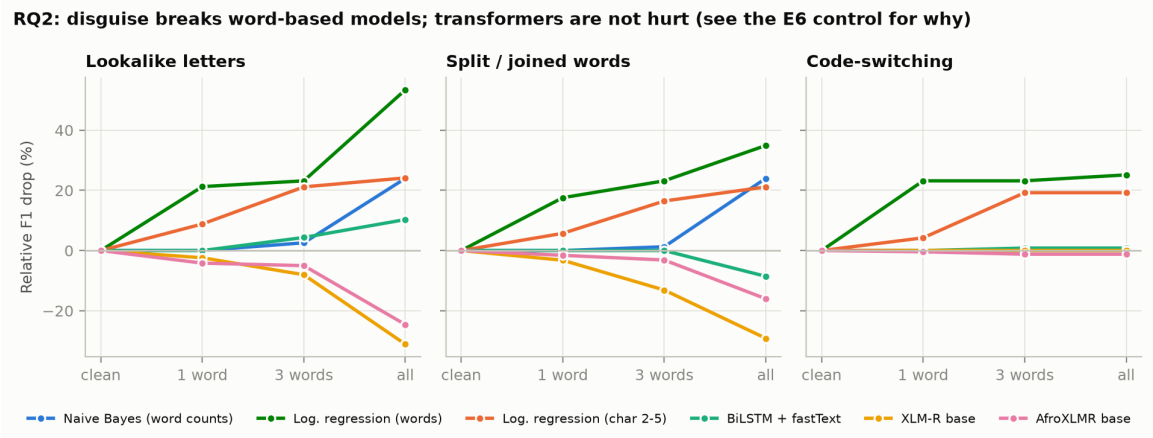


Figure 3. Relative F1 drop by attack and intensity; values below zero mean the attack made the scam easier to catch.



Figure 4. Share of fully disguised scams caught with no defence, with normalisation and with adversarial training; black lines mark recall on the clean test set. Word NB is word-count Naive Bayes and Char LR the character 2-5-gram logistic regression.

5.4 RQ3: Transfer to Chichewa

Zero-shot fraud F1 on the 733 Chichewa messages is 0.46 to 0.75 for trained models, against 0.72 for the nine-word length rule (Figure 5). AfroXLMR ranks Chichewa fraud better than XLM-R (PR-AUC +0.15, $p < 0.001$), consistent with its Chichewa pretraining, but a BiLSTM over Swahili

fastText vectors ranks as well (0.80). With 50 Chichewa examples AfroXLMR reaches 0.87 against XLM-R's 0.78 ($p < 0.001$), and the character LR, with no pretraining at all, reaches 0.88 (difference from AfroXLMR not significant).

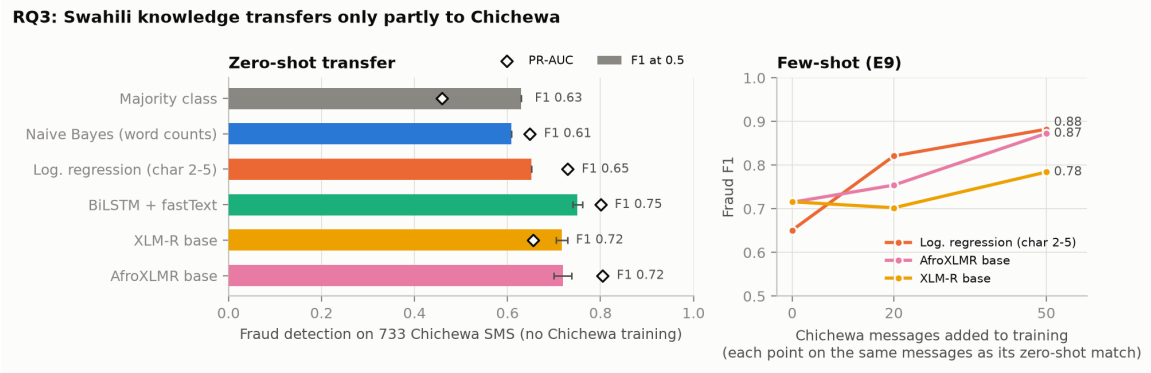


Figure 5. Zero-shot Chichewa fraud F1 and PR-AUC (left), and few-shot F1 scored on the same messages as each zero-shot match (right).

5.5 The Phone-Number Shortcut

Appending a phone number to a genuine chat message makes AfroXLMR call it a scam 96% of the time, and deleting the number from a scam makes it miss 20% (Figure 6). Two transformers of the same size differ widely (XLM-R 40% ± 34% across seeds), so capacity alone does not predict how strongly a model takes the shortcut. Number-balanced training removes the phone-number effect for every model (an appended amount still causes 6% false alarms for the character LR and 21% for character Naive Bayes), raises AfroXLMR's test PR-AUC from 0.958 to 0.985 (the F1 gain, 0.78 to 0.82, is not significant, $p = 0.06$), and lifts the character LR's test F1 from 0.968 to 0.988. The character LR now catches the five landlord messages it missed (scored 0.43 to 0.48 before), at the cost of one missed English job scam and one false alarm on "Naomba unitumie namba ya huyo fundi" (send me the repairman's number). It has costs: lower Chichewa precision for AfroXLMR (-0.06, $p < 0.001$), and most of the character LR's Chichewa transfer lost (0.65 to 0.15), showing that for the classical models "contains a number" was what transferred.

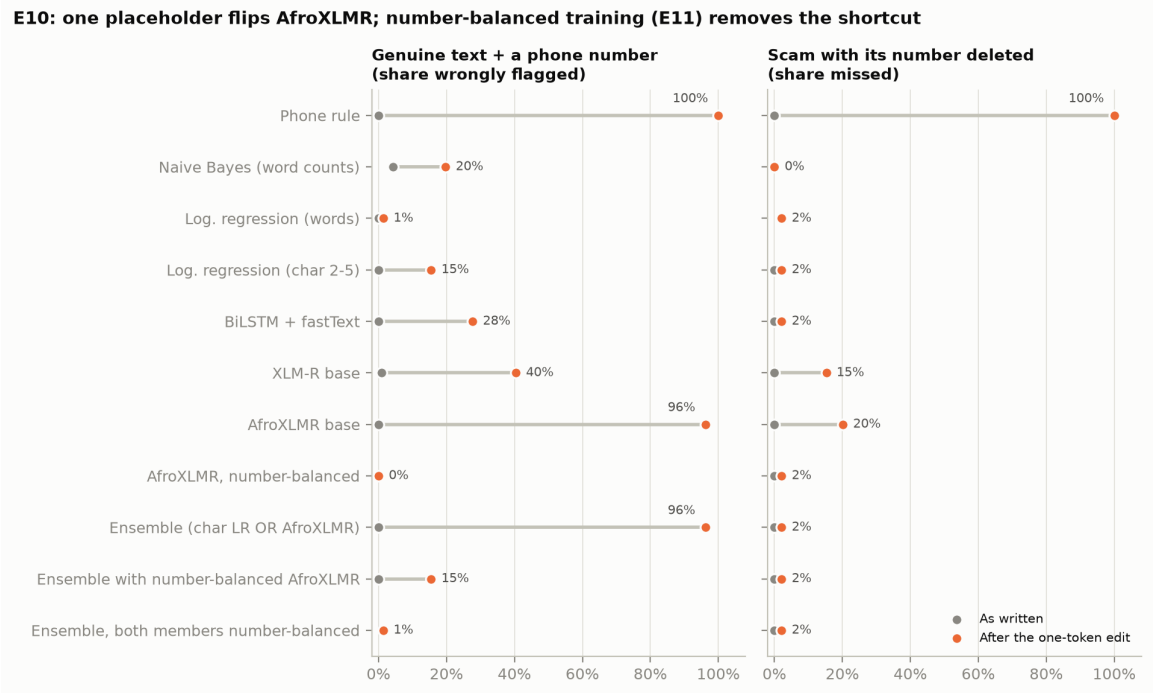


Figure 6. Minimal-pair stress tests (E10): each pair differs by one placeholder, so the gap between the two dots is caused by the number alone.

The character LR and AfroXLMR fail on different messages, so the deployed ensemble flags a message when either flags it. With both members number-balanced it scores test F1 0.994, level with the number-balanced character LR on its own (0.988; +0.006, $p = 0.37$), and stays at 0.95 to 0.99 under lookalike and split-word attacks from either attacker, 0.18 to 0.23 above the character LR there ($p < 0.001$ in the three comparisons tested); the number-balanced character LR on its own falls to 0.50 under the full lookalike attack. Because the idea came from looking at test errors, it was re-checked on two fresh template-disjoint splits never used for design: 0.994 and 0.988, well above AfroXLMR (+0.20 to +0.22, $p < 0.001$) and level with the character LR (+0.02 to +0.03, not significant). Its value is robustness to disguise and to the number shortcut rather than higher clean-text F1.

6. Error Analysis and Discussion

6.1 Successes and Failures

Most errors fall into five kinds, and each points to a cause in the data rather than in the model alone: a social scam with no number, disguised words, service texts the training data never showed, local scam scripts, and messages too short to judge. Table 6 shows representative successes and failures of the deployed models, and Table 7 counts every error of the main runs by kind.

Message (shortened)	Kind	Character LR	AfroXLMR	Verdict
Nitapika wali na mboga leo jioni ... (dinner plans)	Genuine	3%	0%	Passes, correct
The same message + *Nipigie 0712345678*	Genuine with a number	5%	0%	Passes, correct: the shortcut is gone
Mama, nimefika salama Dodoma. Nipigie jioni kwa namba hii 0754123456 tuongee. (arrived safely, call me)	Genuine, number inside a scam phrase	18%	92%	Flagged by AfroXLMR, wrong
The same message without the number	Genuine	19%	0%	Passes, correct
Iyo pesa itume kwenye namba hii 0712345678 jina litakuja JUMA ALLY	Classic scam	100%	100%	Flagged, correct
A "help me" scam written with lookalike letters	Disguised scam	29%	100%	Flagged by AfroXLMR only
The same, with the normalisation defence on	Disguised scam	93%	100%	Flagged by both
Habari za asubuhi. Mimi mwenye nyumba wako hii namba yangu ya Vodacom ...	Landlord impersonation	64%	0%	Flagged by the character LR only
MIRACLE MONEY: lowani mpingo wathu ... imbani phone ku nambala iyi ...	Chichewa scam	26%	100%	Flagged by AfroXLMR only

Table 6. Representative inputs to the deployed number-balanced models (scam probability), as returned by the live app.

Where	Kind of error	NB word	Char LR	BiLSTM	XLM-R	AfroXLMR
Swahili test	Impersonation scam missed	0	5	31	31	31
Swahili test	Genuine text flagged	1	0	0	0	0
Swahili test	Ambiguous short text	2	0	0	0	0
Attacked test (3 attacks, 243 scams)	Disguised scam missed (caught before the disguise)	62	79	2	0	0
Attacked test (3 attacks, 243 scams)	Impersonation scam missed	0	5	31	31	31
Attacked test (3 attacks, 243 scams)	Other scam missed	0	10	56	0	22
Chichewa	Fraud missed	75	149	94	16	113
Chichewa	Telco service text flagged	42	14	16	59	26
Chichewa	Service-like genuine text flagged	59	26	30	90	37
Chichewa	Other genuine text flagged	63	2	12	90	5
Chichewa	Ambiguous short text	96	6	3	11	6

Table 7. Every error of the plain models by kind (seed 42 for neural models). The attacked rows pool the three fully disguised test sets (3 x 81 scams). Kinds are assigned by keyword rules, in order, and then read by hand; a name describes the message, not a proven cause, and a disguised impersonation scam counts as "other" once its words no longer match.

6.2 Why the Models Fail

Impersonation without a number. The landlord script ("this is my new number, why are you quiet?") has no number, amount or link, and the request for money comes later in the conversation. Its words occur only in training scams (*namba yangu* in 49, *mwenye nyumba* in 9), so word Naive Bayes catches all 31 copies and word LR 29. AfroXLMR scores it 0.000, yet 0.985 once a phone number is appended: its decision rests on the <PHONE> token. The BiLSTMs miss nearly all of

it too (at most 4 of 31 in any run). This is shortcut learning in the sense of Geirhos et al. (2020): the models learnt a feature that separated the training classes perfectly but does not define a scam. Number-balanced training helps AfroXLMR a little (6 to 32% of the script caught), which is why the deployed system pairs it with the character LR.

Disguised words. Lookalike letters and invisible characters turn trigger words into unseen tokens, so word Naive Bayes misses 31 of the 81 fully disguised scams under lookalike letters and 31 again under split words, and the character LR newly misses 24 to 29 of 81 under each of the three attacks (scams it caught before the disguise). This matches the fragility of classical models reported by Chiuseni et al. (2026) and the character attacks of Eger et al. (2019) and Boucher et al. (2022). Normalisation repairs the characters it knows, much as a word-recognition front end does (Pruthi et al., 2019), but it cannot repair code-switching or characters outside its table. The transformers miss almost nothing here, but the control sets show they also flag disguised genuine texts: suspicion of odd-looking text, not reading through the disguise.

Service texts flagged in Chichewa. BongoScam contains no genuine bank, telecom or mobile-money service messages, so balance and bundle notices full of amounts look like fraud. AfroXLMR flags 67% ± 20% of the Chichewa telco texts but 17% ± 4% of other genuine Chichewa texts. Taylor and Robert (2025a) likewise found that a telco promotion fooled every model, so this is a property of the task, not only of these models.

Local scam scripts. Some Chichewa frauds follow national scripts that never appear in Tanzanian data: a transporter holding a relative's parcel from South Africa, or relief for flood victims. Only 14 of the 116 Chichewa frauds AfroXLMR misses (Table 7 counts three short ones as ambiguous) are such local schemes, though; most are ordinary money requests in an unseen language, which

is the zero-shot gap that transfer studies describe (Lauscher et al., 2020) and that 20 to 50 Chichewa examples largely close.

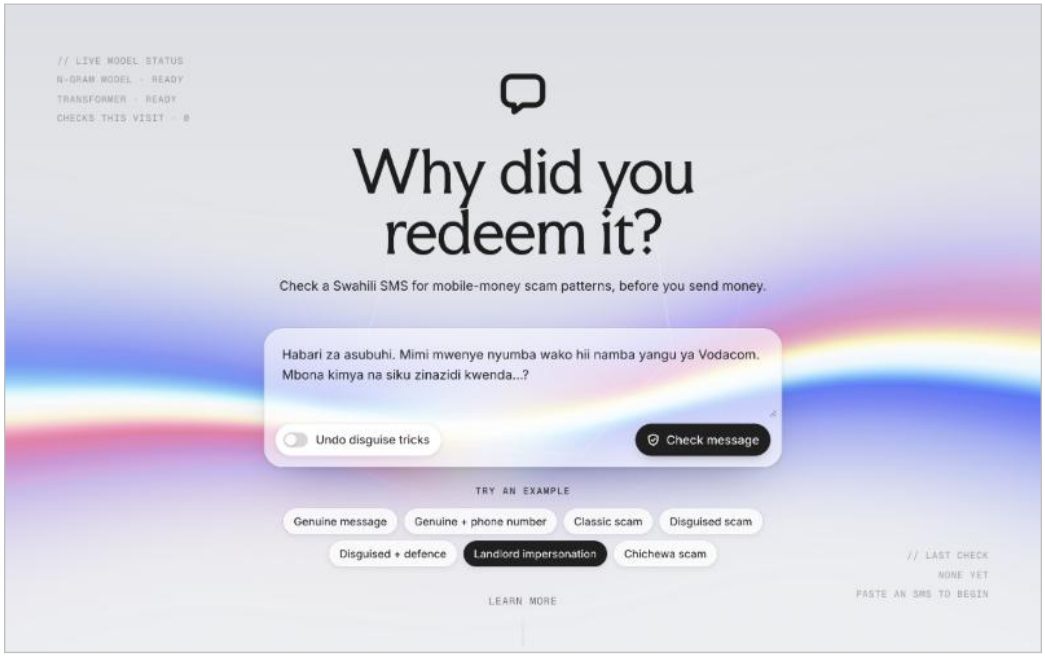
Ambiguous short texts. Messages such as *Chonde ndiyimbileni pa <PHONE>* (please call me on <PHONE>) cannot be judged without the sender or the conversation; Naive Bayes gets 96 of them wrong in Chichewa (95 false alarms) and the other models few.

The near-perfect in-language scores of earlier work are real on random splits of this data, but they partly measure a dataset artefact. A detector for real inboxes needs genuine service messages in training, tests that remove the artefact, and either sender metadata or conversation context for social scams. This differs from the reading of Njame et al. (2026), who take stable ablations as evidence of a linguistic signal. Their folds are random, so a script seen in training can reappear in testing, and removing a feature does not show what a model does when only that feature changes; the minimal-pair tests here change one token and hold the rest of the message fixed.

7. Deployment

The deployed E12 ensemble runs entirely in the visitor's browser on a free static Hugging Face Space (https://huggingface.co/spaces/supserrr/wdyri), so nothing a user types leaves their device. A user pastes an SMS or picks one of seven built-in examples and presses Check (Figures 7 and 8). The app shows the verdict, each model's scam probability against its threshold, the words that pushed each score up or down, and the masked text the models actually read.

(a)



(b)

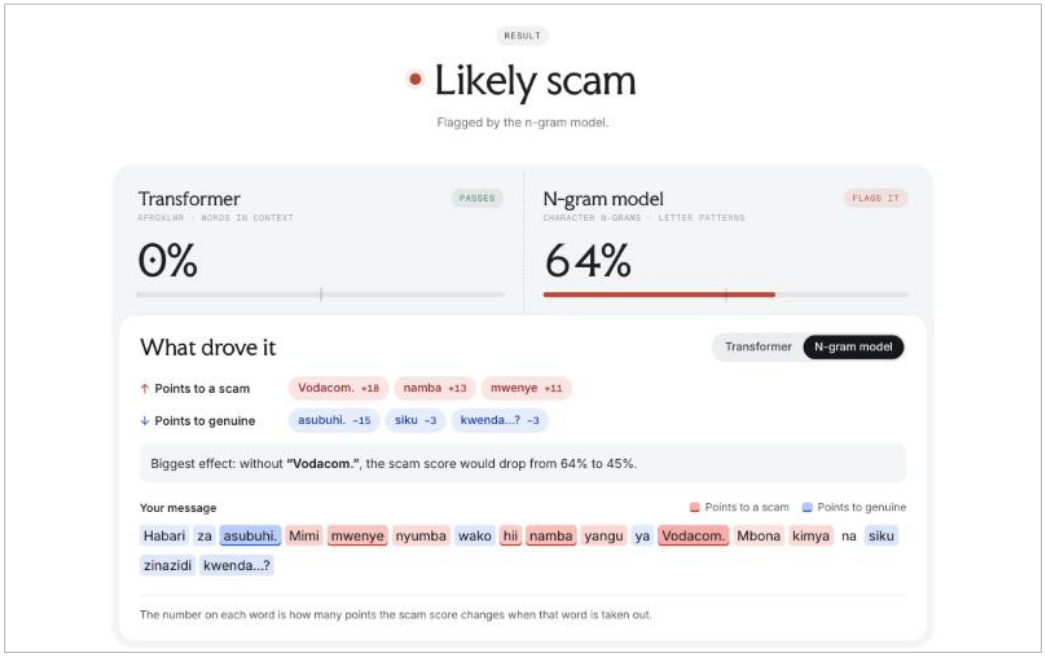


Figure 7. The live app on a desktop screen: (a) the landlord-impersonation example in the text box; (b) its result, flagged by the character n-gram model (64%) while AfroXLMR passes it (0%), with the words that drove the n-gram score.

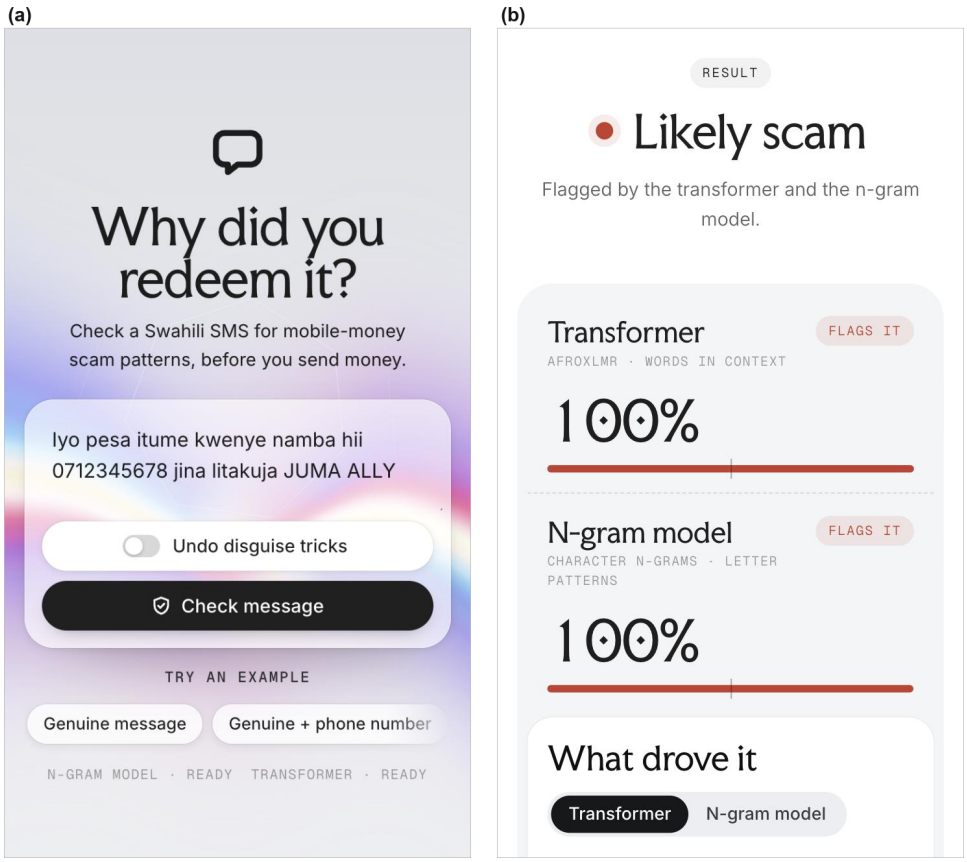


Figure 8. The live app on a 375 x 667 phone screen: (a) a classic scam pasted in; (b) its result, flagged by both models.

The app loads two models. The fine-tuned, number-balanced AfroXLMR (seed 42) is published on the Hugging Face Hub (supserrr/wdyri-afroxlmr) and runs through Transformers.js (Hugging Face, 2026) on ONNX Runtime Web (ONNX Runtime developers, 2026). The number-balanced character LR is exported as its vocabulary, inverse document frequencies and weights, and a short JavaScript module recomputes scikit-learn's character n-gram TF-IDF and the logistic function exactly. A message is flagged if either model's probability reaches its validation-chosen threshold. Word highlights come from leave-one-word-out occlusion, the same method that picks the attacks' trigger words.

Standard 8-bit quantisation broke the transformer (74% decision agreement with PyTorch), because RoBERTa-family activations have outlier channels. The export instead stores the

embedding table in 8 bits and the other weights in 16-bit floats, computing in 32-bit floats (1.1 GB to 364 MB); it agrees with PyTorch on 99.7% of 1,460 evaluation messages and on 100% of the Swahili test set. A JavaScript port of preprocess() spells out Python's Unicode rules for word characters, digits, word boundaries and whitespace, and an automatic check confirms it matches Python on all 4,487 distinct raw and evaluation texts, with the n-gram model agreeing on all 6,622 decisions (largest probability difference 5e-7).

Among plain and number-balanced members, the validation stress pairs favoured both number-balanced (0% false alarms and 0% misses, against 10% false alarms with a plain character LR and 96% with a plain AfroXLMR); the test set played no part in the choice. When the app was deployed, Hugging Face refused a free Gradio Space for this account and asked for a paid plan, while static Spaces are free and do not sleep. The same ensemble also runs as a Gradio app (Abid et al., 2019) locally and in a Colab notebook that prints a public link, the fallback if the Space is unavailable. The first visit downloads the 364 MB model once; the n-gram model answers immediately meanwhile.

8. Limitations and Future Work

8.1 Limitations

Dataset. In-language scores come from 1,064 messages of undocumented origin that two simple shortcuts nearly separate, so they overstate real-world performance. BongoScam's genuine texts read like composed personal chat and contain no service messages, and the Chichewa fraud set is partly rewritten by its authors, with the originals unmarked.

Evaluation. Validation F1 saturates, so most neural comparisons rest on one template-disjoint test set of 152 messages dominated by one script. Per-template views, bootstrap intervals, significance

tests and two fresh splits for the transformer and the ensemble reduce this, but the neural models were not run on ten re-drawn splits, the fresh-split runs use one seed, and the 29 significance tests carry no correction for multiple comparisons. The minimal-pair tests append a bare <PHONE> to the end of a message, so they measure that edit only: a number inside the scam hook "kwa namba hii" still makes the number-balanced AfroXLMR flag a genuine text (92%, against 0% without the number; Table 6).

Attacks and defences. The main trigger words come from a character-LR attacker, so the attacks are white-box for that model and transfer attacks for the others. Adversarial training reuses the attack code, and the normalisation table covers the attack's characters, so E7 measures robustness to known tricks; on held-out lookalike characters normalisation recovers nothing.

Choices informed by test results. The ensemble idea came from test-set errors, and one change to the classical tuning protocol followed a test result. Fresh splits and validation-based member selection reduce, but do not remove, that bias. The masking rules were written after looking at samples of both datasets.

Context and compute. Single messages carry no sender or conversation history, which social scams such as the landlord script need. All models were trained on a laptop CPU, which ruled out a learning-rate search and larger models, and the browser model is a heavy 364 MB first download on a phone.

8.2 Future Work

The most useful next step is data: genuine bank, telecom and mobile-money service messages and real inbox data, collected with documented consent and labelling, so the shortcut cannot form. Every neural model should then be run on repeated template-disjoint splits with a learning-rate

search on a GPU. The hand-written normalisation table should give way to a full Unicode confusables list, tested against attackers that adapt to the defence. Fine-tuning on a few hundred Chichewa messages, and extending transfer to other East and Southern African languages, would test how far the approach travels. Sender metadata or conversation context would address impersonation scams, and distilling the transformer into a smaller model would make the app quicker to load on phones.

9. Conclusion

Near-perfect Swahili scam detection is partly an artefact: the published 98.7% reproduces, but the models largely read whether a message contains a phone number, and that shortcut can be measured and removed. On leakage (RQ1), keeping templates out of the test costs character models 2.6 to 5.7 F1 points and word models about one, while one unseen landlord script with no number drops every neural model to about 0.76 to 0.82 test F1. On robustness (RQ2), lookalike letters halve a word model's F1; normalisation recovers it only for the characters it was written for, adversarial training helps against code-switching, and transformers resist disguise mainly by flagging odd-looking text. On transfer (RQ3), zero-shot Chichewa F1 is 0.46 to 0.75; AfroXLMR ranks fraud better than XLM-R, but 20 to 50 Chichewa examples matter more than the choice of pretrained model.

Appending a number to a genuine message made AfroXLMR flag it 96% of the time, and number-balanced training brought that to 0%. A small ensemble of a character n-gram model and AfroXLMR, both trained number-balanced, scores test F1 0.994, holds 0.95 to 0.99 under disguise, and is the deployed system. The main lesson is that a more complex model is not automatically better: on this data the transformers took the shortcut most strongly, and the most useful steps were

in the evaluation, not the architecture. Leakage-safe splits, minimal-pair tests and control sets showed what aggregate accuracy hid.

References

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). *Gradio: Hassle-free sharing and testing of ML models in the wild* (arXiv:1906.02569). arXiv. https://arxiv.org/abs/1906.02569

Alabi, J. O., Adelani, D. I., Mosbach, M., & Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. In *Proceedings of the 29th International Conference on Computational Linguistics* (pp. 4336–4349). https://aclanthology.org/2022.coling-1.382

Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259–262). https://doi.org/10.1145/2034691.2034742

Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics, 5*, 135–146. https://doi.org/10.1162/tacl_a_00051

Boucher, N., Shumailov, I., Anderson, R., & Papernot, N. (2022). Bad characters: Imperceptible NLP attacks. In *2022 IEEE Symposium on Security and Privacy* (pp. 1987–2004). https://doi.org/10.1109/SP46214.2022.9833641

Chiuseni, D., Bahizire, A., Hama, S., & Ndibwile, J. D. (2026). *Adversarial robustness in smishing detection: A comparative analysis of adversarial fragility in classical vs. transformer-based detection systems* (arXiv:2608.12889). arXiv. https://arxiv.org/abs/2608.12889

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440–8451). https://doi.org/10.18653/v1/2020.acl-main.747

Dioniz, H. (2024a). *Swahili SMS detection dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/henrydioniz/swahili-sms-detection-dataset

Dioniz, H. (2024b). *BongoScam: Ni tumie kwa namba hii SMS detection with machine learning* [Computer software]. GitHub. https://github.com/Henryle-hd/BongoScamDetection

Dodge, J., Ilharco, G., Schwartz, R., Farhadi, A., Hajishirzi, H., & Smith, N. (2020). *Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping* (arXiv:2002.06305). arXiv. https://arxiv.org/abs/2002.06305

Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1383–1392). https://doi.org/10.18653/v1/P18-1128

Ebrahimi, J., Rao, A., Lowd, D., & Dou, D. (2018). HotFlip: White-box adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 31–36). https://doi.org/10.18653/v1/P18-2006

Eger, S., Şahin, G. G., Rücklé, A., Lee, J.-U., Schulz, C., Mesgar, M., Swarnkar, K., Simpson, E., & Gurevych, I. (2019). Text processing like humans do: Visually attacking and shielding NLP systems. In *Proceedings of NAACL-HLT 2019* (pp. 1634–1647). https://aclanthology.org/N19-1165

Elangovan, A., He, J., & Verspoor, K. (2021). Memorization vs. generalization: Quantifying data leakage in NLP performance evaluation. In *Proceedings of the 16th Conference of the European Chapter of the ACL* (pp. 1325–1335). https://aclanthology.org/2021.eacl-main.113

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence, 2*(11), 665–673. https://doi.org/10.1038/s42256-020-00257-z

Gorman, K., & Bedrick, S. (2019). We need to talk about standard splits. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 2786–2791). https://doi.org/10.18653/v1/P19-1267

Grave, E., Bojanowski, P., Gupta, P., Joulin, A., & Mikolov, T. (2018). Learning word vectors for 157 languages. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. https://aclanthology.org/L18-1550

Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S. R., & Smith, N. A. (2018). Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT 2018, Volume 2 (Short Papers)* (pp. 107–112). https://aclanthology.org/N18-2017

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hu, J., Ruder, S., Siddhant, A., Neubig, G., Firat, O., & Johnson, M. (2020). XTREME: A massively multilingual multi-task benchmark for evaluating cross-lingual generalisation. In *Proceedings of the 37th International Conference on Machine Learning* (PMLR 119, pp. 4411–4421). https://proceedings.mlr.press/v119/hu20b.html

Hugging Face. (2026). *Transformers.js* (Version 4.3.0) [Computer software]. https://github.com/huggingface/transformers.js

Jin, D., Jin, Z., Zhou, J. T., & Szolovits, P. (2020). Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. *Proceedings of the AAAI Conference on Artificial Intelligence, 34*(05), 8018–8025. https://doi.org/10.1609/aaai.v34i05.6311

Joshi, P., Santy, S., Budhiraja, A., Bali, K., & Choudhury, M. (2020). The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 6282–6293). https://doi.org/10.18653/v1/2020.acl-main.560

Kaushik, D., Hovy, E., & Lipton, Z. C. (2020). Learning the difference that makes a difference with counterfactually-augmented data. In *International Conference on Learning Representations (ICLR 2020)*. https://arxiv.org/abs/1909.12434

Lauscher, A., Ravishankar, V., Vulić, I., & Glavaš, G. (2020). From zero to hero: On the limitations of zero-shot language transfer with multilingual transformers. In *Proceedings of EMNLP 2020* (pp. 4483–4499). https://aclanthology.org/2020.emnlp-main.363

Mambina, I. S., Ndibwile, J. D., & Michael, K. F. (2022). Classifying Swahili smishing attacks for mobile money users: A machine-learning approach. *IEEE Access, 10*, 83061–83074. https://doi.org/10.1109/ACCESS.2022.3196464

Mambina, I. S., Ndibwile, J. D., Uwimpuhwe, D., & Michael, K. F. (2024). Uncovering SMS spam in Swahili text using deep learning approaches. *IEEE Access, 12*, 25164–25175. https://doi.org/10.1109/ACCESS.2024.3365193

McCoy, R. T., Pavlick, E., & Linzen, T. (2019). Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3428–3448). https://aclanthology.org/P19-1334

Mosbach, M., Andriushchenko, M., & Klakow, D. (2021). On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines. In *International Conference on Learning Representations (ICLR 2021)*. https://arxiv.org/abs/2006.04884

Njame, R. A., Sanga, G., & Tende, I. (2026). A machine-learning model for phishing detection in Swahili messages: A case of Tanzania. *East African Journal of Information Technology, 9*(2), 97–114. https://doi.org/10.37284/eajit.9.2.5655

Ogueji, K., Zhu, Y., & Lin, J. (2021). Small data? No problem! Exploring the viability of pretrained multilingual language models for low-resourced languages. In *Proceedings of the 1st Workshop on Multilingual Representation Learning* (pp. 116–126). https://doi.org/10.18653/v1/2021.mrl-1.11

ONNX Runtime developers. (2026). *ONNX Runtime* (Version 1.30.0) and ONNX Runtime Web (Version 1.31.0-dev.20260914) [Computer software]. https://onnxruntime.ai

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). PyTorch: An imperative style,

high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). https://arxiv.org/abs/1912.01703

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Pires, T., Schlinger, E., & Garrette, D. (2019). How multilingual is multilingual BERT? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 4996–5001). https://doi.org/10.18653/v1/P19-1493

Pruthi, D., Dhingra, B., & Lipton, Z. C. (2019). Combating adversarial misspellings with robust word recognition. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 5582–5591). https://doi.org/10.18653/v1/P19-1561

Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks* (pp. 45–50). ELRA.

Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4902–4912). https://aclanthology.org/2020.acl-main.442

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing, 45*(11), 2673–2681. https://doi.org/10.1109/78.650093

Søgaard, A., Ebert, S., Bastings, J., & Filippova, K. (2021). We need to talk about random splits. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume* (pp. 1823–1832). https://doi.org/10.18653/v1/2021.eacl-main.156

Tanzania Communications Regulatory Authority. (2026). *Communications sector performance report: Quarter ending June 2026*. https://www.tcra.go.tz/tcra-tovuti/2026/mamlaka-website/documents/quarterly-statistics/Communications_Sector_performance_report_for_quarter_ending_June_2026_ad44159230.pdf

Taylor, A., & Robert, A. (2025a). Using machine learning to detect fraudulent SMSs in Chichewa. In *Integrating AI in Science, Management, and Technology (AISMT 2025)* (Communications in Computer and Information Science, Vol. 2699, pp. 143–169). Springer. https://doi.org/10.1007/978-3-032-08260-2_12

Taylor, A., & Robert, A. (2025b). *SMS fraud classification dataset for Chichewa* [Data set]. Zenodo. https://doi.org/10.5281/zenodo.14607454

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le

Scao, T., Gugger, S., . . . Rush, A. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations* (pp. 38–45). https://doi.org/10.18653/v1/2020.emnlp-demos.6

Wu, S., & Dredze, M. (2019). Beto, Bentz, Becas: The surprising cross-lingual effectiveness of BERT. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (pp. 833–844). https://doi.org/10.18653/v1/D19-1077

Appendix A. Significance Tests

Comparison	Metric	Difference	95% interval	p
AfroXLMR vs XLM-R, Chichewa ranking	PR-AUC	+0.150	+0.080 to +0.243	< 0.001
AfroXLMR vs XLM-R, Swahili test F1	F1	+0.016	+0.000 to +0.065	0.387
AfroXLMR vs XLM-R, Chichewa F1	F1	+0.002	-0.041 to +0.044	0.948
Plain ensemble vs AfroXLMR, Swahili test F1	F1	+0.193	+0.118 to +0.280	< 0.001
Plain ensemble vs character LR, full lookalike attack F1	F1	+0.231	+0.142 to +0.328	< 0.001
Plain ensemble vs character LR, full structural attack F1	F1	+0.135	+0.048 to +0.255	< 0.001
Normalisation vs none, word LR under full lookalike	F1	+0.517	+0.401 to +0.636	< 0.001
Normalisation vs none, word LR under held-out lookalikes	F1	+0.000	+0.000 to +0.000	1.000
Number-balanced vs plain AfroXLMR, Chichewa precision	Precision	-0.064	-0.122 to -0.024	< 0.001
Number-balanced vs plain AfroXLMR, Chichewa F1	F1	-0.003	-0.050 to +0.034	0.993
Number-balanced vs plain AfroXLMR, Swahili test F1	F1	+0.044	+0.000 to +0.126	0.059
Number-balanced vs plain character LR, Chichewa F1	F1	-0.505	-0.561 to -0.444	< 0.001
AfroXLMR +50 Chichewa vs XLM-R +50, F1	F1	+0.088	+0.052 to +0.134	< 0.001
Character LR +50 Chichewa vs AfroXLMR +50, F1	F1	+0.010	-0.017 to +0.035	0.480
Deployed ensemble vs number-balanced AfroXLMR, test F1	F1	+0.174	+0.093 to +0.272	< 0.001
Deployed ensemble vs character LR, Swahili test F1	F1	+0.026	+0.000 to +0.060	0.047
Deployed ensemble vs number-balanced character LR, Swahili test F1	F1	+0.006	+0.000 to +0.020	0.373
Deployed ensemble vs number-balanced character LR, full lookalike attack F1	F1	+0.468	+0.340 to +0.599	< 0.001

Comparison	Metric	Difference	95% interval	p
Deployed ensemble vs character LR, full lookalike attack F1	F1	+0.229	+0.126 to +0.333	< 0.001
Deployed ensemble vs character LR, lookalike attack (NB attacker)	F1	+0.218	+0.144 to +0.304	< 0.001
Deployed ensemble vs character LR, structural attack (NB attacker)	F1	+0.222	+0.149 to +0.308	< 0.001
Deployed ensemble vs character LR, fresh split 1 test F1	F1	+0.019	-0.006 to +0.049	0.105
Deployed ensemble vs character LR, fresh split 2 test F1	F1	+0.026	-0.000 to +0.060	0.051
Deployed ensemble vs AfroXLMR, fresh split 1 test F1	F1	+0.203	+0.132 to +0.288	< 0.001
Deployed ensemble vs AfroXLMR, fresh split 2 test F1	F1	+0.224	+0.149 to +0.308	< 0.001
Random vs template split, Naive Bayes word counts	F1	+0.010	Mann-Whitney U	0.011
Random vs template split, Naive Bayes char	F1	+0.057	Mann-Whitney U	< 0.001
Random vs template split, logistic regression words	F1	+0.006	Mann-Whitney U	0.018
Random vs template split, logistic regression char	F1	+0.026	Mann-Whitney U	< 0.001

Table A1. Key differences: first system minus second. Paired bootstrap with 1,000 resamples of messages per seed pair, pooled; the last four rows compare ten re-drawn splits of each type with a Mann-Whitney U test. No correction for multiple comparisons is applied to the 29 tests, so p-values near 0.05 are weak evidence.